Supplier Disruption Detection + Replenishment Agents

Lakshya Sharma · Track A (Agentic / Multi-Agent AI) · Scenario S1 (Supply Chain Risk) Repo: github.com/lakshya082001/FMs-AgenticAI

1. Problem & Domain

- **Task:** S1 Supply Chain Risk, Track A.
- **Problem:** detect supplier disruption early, recommend wait / expedite / switch supplier, human signs off on risky plans.
- **Why not other Problem:** Recommendation could have been a great problem but (Better data and SLM/model training requires too much time), Documents(RAG) is a common task and track, but supply chain with FMs+Agents is a path that can be extended to a larger business set.
- **Why not other tracks:** B has no language task to fine-tune. C stops at a risk score, no action. D has no document corpus.
- **Why agentic:** needs tools, independent plan check, human gate. Single LLM or classifier gives none.

Flow

```
Supplier-day row -> Orchestrator (code router)
  -> Risk Analyst (TabPFN)        p < threshold -> end: "not at risk"
  -> Exposure Analyst (at-risk SKUs)
  -> Planner (wait / expedite / switch)
  -> Check (code, retry once)
  -> Human gate (borderline p, cost > 10,000, failed check)
  -> Action card
```

2. Approach & Algorithm Decisions

Why a model, not agents alone

- Feature relations (late rates, in-transit delays, volume) need a graph DB or a model.
- Graph DB: needs exhaustive data, more time. Not now.
- Chose TabPFN (tabular foundation model) as quick placeholder.
- TabPFN: instant predictions, small/medium data, no hyperparameter tuning.

Decisions

Decision	Chosen	Rejected
Detector	TabPFN, 11 trailing-window features	Graph DB (time). XGBoost (not run, no claim). LSTM/Chronos (3.6K rows)
Data	Real DataCo orders + simulated disruptions	Raw labels (no temporal signal). Fully synthetic (circular)

Decision	Chosen	Rejected
Framework	LangGraph (state, retry loop, native interrupt)	CrewAI, AutoGen (weaker fit for retry + gate)
Decomposition	3 LLM agents + code router + code check	One LLM with tools (no verification). LLM critic (code check cheaper)
Planner	LLM picks from tool-computed costs	Pure argmin (no rationale). Check keeps LLM within 10% of cheapest

Agents

Agent	One-liner
Orchestrator (code)	Routes each step, logs reason
Risk Analyst	Validates row, runs TabPFN, writes hedged summary
Exposure Analyst	Lists SKUs with cover below disrupted lead time
Planner	Picks wait / expedite / switch per SKU

Silent failures handled

- LLM said "0.27 below 0.25": status now computed in code.
- Missed cheapest option on 3 of 15 decisions: optimality check + retry.
- Wrong numbers: number check. Unresolved: human.

Related work

- arXiv 2601.09680: 7 CrewAI agents, news-triggered. Neo4j multi-tier graph, rule-weighted risk score.
- Neo4j article: agents query supply-chain graph DB. Full text unread (blocked), from description.
- **Ours:** model on structured data instead of graph.

3. Results & Error Analysis

Data

- DataCo orders + simulated suppliers (8), SKUs (118), disruption episodes.
- Target: supplier in disruption episode that day.
- Features: late rates (3/7/14/30/90d), 7d volume, deviations, days since alert, in-transit overdue count + share.
- Split (temporal): train 3,656, val 1,472, test 2,184.

Metrics

	AUC	Avg prec.	Brier	ECE	Base rate	Best single feature AUC
Val	0.659	0.298	0.121	0.071	0.145	0.689
Test	0.652	0.402	0.172	0.091	0.234	0.664

- Threshold 0.25: 27/35 test episodes caught, median delay 5 days, false-alarm rate 17.4%.
- TabPFN does not beat one feature (`od_share`). Gain is from the in-transit signal.
- One seed, 35 episodes: ~0.01 AUC is noise.
- Agent run: 40–46 s (~30 s TabPFN on CPU), 1–4 LLM calls, 2–4K tokens.

Failure points

- **Long-range temporal dependence:** Require more feature engineering and comlete base data. This is just a quick mockup data.
- **Train→test shift:** disruption 15% → 23%, calibration worse.
- **Detection lag:** Current model does the prediction over rolled up temporal features without taking previous supplier timelines directly as input as TabPFN is structured row to label prediction model, can't model on the previous time instances.
- **Assumptions:** alternate suppliers undisrupted, single-tier.

4. Production & Limitations

- **Model:** export weights, host (or hosted API). Daily batch scoring on GPU, cached.
- **Graph (optional):** Neo4j for multi-tier supplier network.
- **Agents:** deploy LangGraph as service.
- **Monitoring:** alert rate, feature drift, calibration.
- **Limitations:** disruptions simulated, so not real-world accuracy. TabPFN is just a quick exploration method but not the only one and it requires license as well. It's limitation is No time series data modelling and currently the absence of supplier feature relation DB
- **Next:**
- what-if agent: which feature change clears an alert.
- Real data+Feature relations(Graph DBs)
- Graph DBs to directly agents like arXiv 2601.09680 or DB to vectorization -> ML modelling -> Agents